NYC Taxi Urban Mobility Data Explorer - Technical Report

Team: **SME INC** | Members: Erica Sheja, Isingizwe Elohim Malaïka, Ishimwe Samuel
Repo: https://github.com/malaika-11/nyc-taxi-mobility-dashboard.git

1. Problem Framing and Dataset Analysis

We analyzed the NYC TLC **Yellow Taxi trip records for January 2019** (a fact table of about 7.5 million trips after cleaning) together with the **taxi_zone_lookup** dimension table (265 zones, borough, service zone) and the **taxi_zones** shapefile (zone polygons). Each trip has pickup/dropoff timestamps, zone IDs, distance, passenger count, rate code, payment type, and itemized fare components. The goal: when and where does the city use taxis, and what do trips cost?

Data challenges: Raw TLC data is messy: zero or negative distances, negative fares (refunds), dropoffs before pickups, trips lasting days, implausible speeds, timestamps outside the month, unknown location IDs, missing passenger counts, and duplicated rows. **build_db.py** applies the rules below in order; the first rule a row breaks is stored in the excluded_record table (source file, row, reason code), so every removal is traceable.

Rule (reason code)	Assumption/threshold	Action
MISSING_VALUE, BAD_LOCATION_ID, UNKNOWN_VENDOR/PAYMENT/RATE_CODE	Required fields present; location IDs in 1-265; codes exist in the TLC data dictionary	Exclude
OUTSIDE_DATE_RANGE, DROPOFF_NOT_AFTER_PICKUP, TRIP_OVER_24_HOURS	Pickup inside the loaded month; duration between 0 and 24 h	Exclude
ZERO_OR_NEGATIVE_DISTANCE, DISTANCE_OVER_LIMIT, FARE_OVER_LIMIT, NEGATIVE_AMOUNT	Distance in (0, 100] miles; fare at most $500; no negative money	Exclude
IMPOSSIBLE_SPEED, DUPLICATE_ROW	Speed at most 100 mph; duplicates detected by a hash of 7 identifying columns (also across chunks)	Exclude
ODD_PASSENGER_COUNT, FAST_TRIP (>70 mph), TOTAL_BELOW_FARE	Plausible but suspicious	Keep, log in suspicious_record

Counts for the report: raw rows [FILL: raw count], kept 7,524,401, excluded [FILL] (run SELECT reason_code, COUNT(*) FROM excluded_record GROUP BY 1 ORDER BY 2 DESC;). **Derived features** (stored in trip / trip_fare): **duration_min** (dropoff - pickup) shows congestion; **avg_speed_mph** (distance/hours) is both a quality check and a congestion proxy; **fare_per_mile** (fare/distance) separates short expensive hops from long cheap ones; **tip_pct** (tip/fare, card payments

only, since cash tips are not recorded); plus **pickup_hour** and **pickup_dow** to make time queries index-friendly.

Unexpected observation. Ranked by total pickups, Thursday is the busiest weekday (1,327,275). But January 2019 has five Thursdays and only four Fridays, so totals hide the real pattern: **Friday has the highest average daily demand (265,749 vs 265,455)**. We therefore chart average pickups per day (per-weekday date counts live in a weekday_average summary table) instead of raw totals.

2. System Architecture and Design Decisions

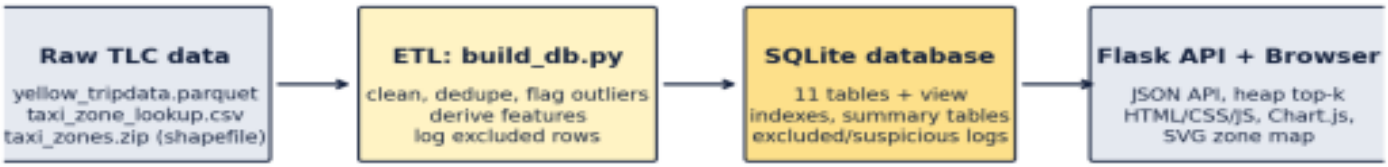


Figure 1. Pipeline from raw TLC files to the browser dashboard.

Stack. Python/pandas for chunked ETL (200k rows at a time, so a 7.5M-row month never sits in memory); **SQLite** because it needs no server, ships as one file, and is fast enough for read-only analytics; **Flask** for a thin JSON API; plain **HTML/CSS/JS** with **Chart.js** and a hand-built SVG choropleth (zone polygons re-projected from EPSG:2263 to lat/lon) so the map needs no tile server or internet. **Schema (3NF).** Lookup tables borough, service_zone, vendor, rate_code, payment_type store each label once; zone references them, and zone_geometry holds the polygons; trip (what/when/where) and trip_fare (money, 1:1 by trip_id) split narrow query columns from wide fare columns; excluded_record and suspicious_record hold the audit log. Indexes cover pickup time, pickup/dropoff zone, (day-of-week, hour), distance, duration, payment type, and fare. View v_trip_full joins everything for the API. **Trade-offs.** (1) Pre-computed summary tables (hourly_demand, daily_demand, zone_stats...) make the dashboard load instantly, at the cost of being stale until rebuilt; the API falls back to live queries if they are missing. (2) Splitting trip_fare costs a join but keeps the hot table small. (3) SQLite does not scale to many concurrent writers; PostgreSQL would be the production choice. (4) We filter strictly (excluding ~[X]% of rows) and log everything, preferring clean analysis over completeness.

3. Algorithmic Logic and Data Structures

Problem. The dashboard must show the top-K busiest zones (by pickups, dropoffs or average fare) and will later need top-K origin-destination routes (up to 265 x 265 = 70k pairs). Sorting everything costs O(n log n) and builds a full sorted copy. We implemented a **binary min-heap** and a **top_k** function from scratch in backend/algorithms.py (no heapq, sorted(), Counter, sort_values).

Approach: keep a heap of the K best rows seen so far; its root is the weakest, so a new row enters only if it beats the root, replacing it in O(log K). The heap is stored in a flat list (children of i are 2i+1, 2i+2) and restored by sift-up/sift-down. Results are emitted smallest-first and reversed manually. It powers /api/zones/top and the map's ranking panel; unit tests compare it to Python's sorting.

```
top_k(rows, K, key): sift_down(i):
 heap = empty min-heap loop: m = i
 for row in rows: if left child < heap[m]: m = left
 if size(heap) < K: push(row) if right child < heap[m]: m = right
 elif row[key] > root: replace_root(row) if m == i: stop
 out = pop all from heap (ascending) swap(i, m); i = m
```

return reverse(out) (descending)

Complexity: n = rows, K = result size. **Time O(n log K)** (each row costs one comparison, and at most one O(log K) replace); the final extraction is O(K log K). **Space O(K)** beyond the input, versus O(n) for a sorted copy. For K = 10 and n = 70,000 routes, the heap does about 70k comparisons and few swaps instead of roughly 1.1M comparisons for a full sort.

4. Insights and Interpretation

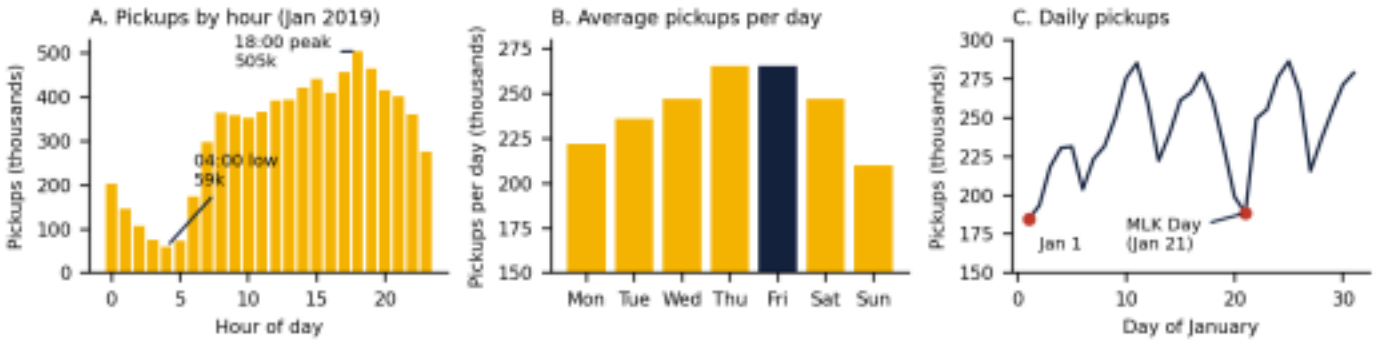


Figure 2. (A) /api/hourly-demand, (B) /api/weekday-average, (C) /api/daily-demand, all from the cleaned January 2019 data (dashboard charts show the same series).

Insight 1 - Demand peaks in the early evening, not the morning rush.

From the hourly counts (GROUP BY pickup_hour), the busiest hour is **18:00 with 505,201 pickups, 8.5x the quietest hour (04:00, 59,184)**. Pickups climb from 72,626 at 05:00 to 365,805 at 08:00, then plateau and climb again to the evening peak. Meaning: taxis serve the commute home, dinner and nightlife more than the commute in; supply (fleet size, driver shifts) should be tuned to 15:00-21:00.

Insight 2 - Friday, not Thursday, is the busiest day once days are counted fairly.

Averaging per calendar day, Friday (265,749) edges out Thursday (265,455), while Sunday is lowest (210,218). Weekdays carry 75.7% of all pickups, but only because there are more of them: per day a weekday averages 247,608 pickups vs 228,676 on weekend days, just **7.6% lower**. Meaning: weekend demand is much stronger than totals suggest, so weekend service levels matter, and fair comparisons need per-day averages.

Insight 3 - Holidays leave a clear fingerprint. The daily series (GROUP BY date) shows two dips: New Year's Day (184,436) and **Martin Luther King Jr. Day (Mon 21 Jan, 188,637)**, **19% below** the average of the other three Mondays (232,992). Meaning: commuter-driven demand collapses on public holidays, so forecasting needs a holiday calendar, and the dataset's anomaly detection should not treat such dips as bad data.

5. Reflection and Future Work

Technical challenges. Memory and speed on 7.5M rows (solved with chunking and cross-chunk duplicate hashing); choosing defensible outlier thresholds (we log rather than silently drop); re-projecting shapefile coordinates from state-plane feet to lat/lon; and keeping the dashboard fast with summary tables. **Team challenges:** [TEAM: describe coordination, merge conflicts, integration of separately built parts, how you resolved them].

Next steps for a real product. Add origin-destination flow analysis and a time slider on the map; load multiple months and years to separate seasonality from one-off events; join weather and holiday data to explain demand; move to PostgreSQL/PostGIS with an incremental, scheduled ETL; add authentication, caching, and automated deployment; and extend the unit tests to the ETL rules.